

Context editing

Automatically manage conversation context as it grows with context editing.

Copy page

⌵

Overview

Context editing allows you to automatically manage conversation context as it grows, helping you optimize costs and stay within context window limits. You can use server-side API strategies, client-side SDK features, or both together.

Approach	Where it runs	Strategies	How it works
Server-side	API	Tool result clearing (<code>clear_tool_uses_20250919</code>) Thinking block clearing (<code>clear_thinking_20251015</code>)	Applied before the prompt reaches Claude. Clears specific content from conversation history. Each strategy can be configured independently.
Client-side	SDK	Compaction	Available in Python and TypeScript SDKs when using <code>tool_runner</code> . Generates a summary and replaces full conversation history. See Compaction below.

Server-side strategies

i

Context editing is currently in beta with support for tool result clearing and thinking block clearing. To enable it, use the beta header `context-management-2025-06-27` in your API requests.

Please reach out through our [feedback form](#) to share your feedback on this feature.

Tool result clearing

The `clear_tool_uses_20250919` strategy clears tool results when conversation context grows beyond your configured threshold. When activated, the API automatically clears the c

calls (the tool use parameters) by setting `clear_tool_inputs` to `true`.

Thinking block clearing

The `clear_thinking_20251015` strategy manages `thinking` blocks in conversations when extended thinking is enabled. This strategy automatically clears older thinking blocks from previous turns.

 **Default behavior:** When extended thinking is enabled without configuring the `clear_thinking_20251015` strategy, the API automatically keeps only the thinking blocks from the last assistant turn (equivalent to `keep: {type: "thinking_turns", value: 1}`).

To maximize cache hits, preserve all thinking blocks by setting `keep: "all"` .

 An assistant conversation turn may include multiple content blocks (e.g. when using tools) and multiple thinking blocks (e.g. with [interleaved thinking](#)).

Context editing happens server-side

Context editing is applied **server-side** before the prompt reaches Claude. Your client application maintains the full, unmodified conversation history—you do not need to sync your client state with the edited version. Continue managing your full conversation history locally as you normally would.

Context editing and prompt caching

Context editing's interaction with [prompt caching](#) varies by strategy:

- **Tool result clearing:** Invalidates cached prompt prefixes when content is cleared. To account for this, we recommend clearing enough tokens to make the cache invalidation worthwhile. Use the `clear_at_least` parameter to ensure a minimum number of tokens is cleared each time. You'll incur cache write costs each time content is cleared, but subsequent requests can reuse the newly cached prefix.
- **Thinking block clearing:** When thinking blocks are **kept** in context (not cleared), the prompt cache is preserved, enabling cache hits and reducing input token costs. When thinking blocks are **cleared**, the cache is invalidated at the point where clearing occurs. Configure the `keep` parameter based on whether you want to prioritize cache performance or context window availability.

Supported models

Context editing is available on:

- Claude Opus 4 (`claude-opus-4-20250514`)
- Claude Sonnet 4.5 (`claude-sonnet-4-5-20250929`)
- Claude Sonnet 4 (`claude-sonnet-4-20250514`)
- Claude Haiku 4.5 (`claude-haiku-4-5-20251001`)

Tool result clearing usage

The simplest way to enable tool result clearing is to specify only the strategy type, as all other [configuration options](#) will use their default values:

cURL ▾



```
curl https://api.anthropic.com/v1/messages \
  --header "x-api-key: $ANTHROPIC_API_KEY" \
  --header "anthropic-version: 2023-06-01" \
  --header "content-type: application/json" \
  --header "anthropic-beta: context-management-2025-06-27" \
  --data '{
    "model": "claude-sonnet-4-5",
    "max_tokens": 4096,
    "messages": [
      {
        "role": "user",
        "content": "Search for recent developments in AI"
      }
    ],
    "tools": [
      {
        "type": "web_search_20250305",
        "name": "web_search"
      }
    ],
    "context_management": {
      "edits": [
        {"type": "clear_tool_uses_20250919"}
      ]
    }
  }'
```

Advanced configuration

cURL ▾



```
--header "anthropic-version: 2023-06-01" \  
--header "content-type: application/json" \  
--header "anthropic-beta: context-management-2025-06-27" \  
--data '{  
  "model": "claude-sonnet-4-5",  
  "max_tokens": 4096,  
  "messages": [  
    {  
      "role": "user",  
      "content": "Create a simple command line calculator app using Python"  
    }  
  ],  
  "tools": [  
    {  
      "type": "text_editor_20250728",  
      "name": "str_replace_based_edit_tool",  
      "max_characters": 10000  
    },  
    {  
      "type": "web_search_20250305",  
      "name": "web_search",  
      "max_uses": 3  
    }  
  ],  
  "context_management": {  
    "edits": [  
      {  
        "type": "clear_tool_uses_20250919",  
        "trigger": {  
          "type": "input_tokens",  
          "value": 30000  
        },  
        "keep": {  
          "type": "tool_uses",  
          "value": 3  
        },  
        "clear_at_least": {  
          "type": "input_tokens",  
          "value": 5000  
        },  
        "exclude_tools": ["web_search"]  
      }  
    ]  
  }  
}'
```

Enable thinking block clearing to manage context and prompt caching effectively when extended thinking is enabled:

cURL ▾

```
curl https://api.anthropic.com/v1/messages \
--header "x-api-key: $ANTHROPIC_API_KEY" \
--header "anthropic-version: 2023-06-01" \
--header "content-type: application/json" \
--header "anthropic-beta: context-management-2025-06-27" \
--data '{
  "model": "claude-sonnet-4-5-20250929",
  "max_tokens": 1024,
  "messages": [...],
  "thinking": {
    "type": "enabled",
    "budget_tokens": 10000
  },
  "context_management": {
    "edits": [
      {
        "type": "clear_thinking_20251015",
        "keep": {
          "type": "thinking_turns",
          "value": 2
        }
      }
    ]
  }
}'
```

Configuration options for thinking block clearing

The `clear_thinking_20251015` strategy supports the following configuration:

Configuration option	Default	Description
keep	<code>{type: "thinking_turns", value: 1}</code>	Defines how many recent assistant turns with thinking blocks to preserve. Use <code>{type: "thinking_turns", value: N}</code> where N must be > 0 to keep the last N turns, or <code>"all"</code> to keep all thinking blocks.

```
// Keep thinking blocks from the last 3 assistant turns
{
  "type": "clear_thinking_20251015",
  "keep": {
    "type": "thinking_turns",
    "value": 3
  }
}

// Keep all thinking blocks (maximizes cache hits)
{
  "type": "clear_thinking_20251015",
  "keep": "all"
}
```

Combining strategies

You can use both thinking block clearing and tool result clearing together:

 When using multiple strategies, the `clear_thinking_20251015` strategy must be listed first in the `edits` array.

Python 



```
max_tokens=1024,
messages=[...],
thinking={
  "type": "enabled",
  "budget_tokens": 10000
},
tools=[...],
betas=["context-management-2025-06-27"],
context_management={
  "edits": [
    {
      "type": "clear_thinking_20251015",
      "keep": {
        "type": "thinking_turns",
        "value": 2
      }
    },
    {
      "type": "clear_tool_uses_20250919",
      "trigger": {
        "type": "input_tokens",
        "value": 50000
      },
      "keep": {
        "type": "tool_uses",
        "value": 5
      }
    }
  ]
}
}
```

Configuration options for tool result clearing

Configuration option	Default	Description
trigger	100,000 input tokens	Defines when the context editing strategy activates. Once the prompt exceeds this threshold, clearing will begin. You can specify this value in either <code>input_tokens</code> or <code>tool_uses</code> .
keep	3 tool uses	Defines how many recent tool use/result pairs to keep after clearing occurs. The API removes the oldest tool interactions first, preserving the most recent ones.

<code>clear_at_least</code>	None	activates. If the API can't clear at least the specified amount, the strategy will not be applied. This helps determine if context clearing is worth breaking your prompt cache.
<code>exclude_tools</code>	None	List of tool names whose tool uses and results should never be cleared. Useful for preserving important context.
<code>clear_tool_inputs</code>	false	Controls whether the tool call parameters are cleared along with the tool results. By default, only the tool results are cleared while keeping Claude's original tool calls visible.

Context editing response

You can see which context edits were applied to your request using the `context_management` response field, along with helpful statistics about the content and input tokens cleared.

Response 

```
{
  "id": "msg_013Zva2CMHLNnXjNJKqJ2EF",
  "type": "message",
  "role": "assistant",
  "content": [...],
  "usage": {...},
  "context_management": {
    "applied_edits": [
      // When using `clear_thinking_20251015`
      {
        "type": "clear_thinking_20251015",
        "cleared_thinking_turns": 3,
        "cleared_input_tokens": 15000
      },
      // When using `clear_tool_uses_20250919`
      {
        "type": "clear_tool_uses_20250919",
        "cleared_tool_uses": 8,
        "cleared_input_tokens": 50000
      }
    ]
  }
}
```

For streaming responses, the context edits will be included in the final `message_delta` event:

```
{
  "type": "message_delta",
  "delta": {
    "stop_reason": "end_turn",
    "stop_sequence": null
  },
  "usage": {
    "output_tokens": 1024
  },
  "context_management": {
    "applied_edits": [...]
  }
}
```

Token counting

The [token counting](#) endpoint supports context management, allowing you to preview how many tokens your prompt will use after context editing is applied.

cURL ▾



```
--header "anthropic-version: 2023-06-01" \  
--header "content-type: application/json" \  
--header "anthropic-beta: context-management-2025-06-27" \  
--data '{  
  "model": "claude-sonnet-4-5",  
  "messages": [  
    {  
      "role": "user",  
      "content": "Continue our conversation..."  
    }  
  ],  
  "tools": [...],  
  "context_management": {  
    "edits": [  
      {  
        "type": "clear_tool_uses_20250919",  
        "trigger": {  
          "type": "input_tokens",  
          "value": 30000  
        },  
        "keep": {  
          "type": "tool_uses",  
          "value": 5  
        }  
      }  
    ]  
  }  
}'
```

Response



```
{  
  "input_tokens": 25000,  
  "context_management": {  
    "original_input_tokens": 70000  
  }  
}
```

The response shows both the final token count after context management is applied (`input_tokens`) and the original token count before any clearing occurred (`original_input_tokens`).

Using with the Memory Tool

information. This enables Claude to save tool results or context to its memory files before they're cleared from the conversation history.

This combination allows you to:

- **Preserve important context:** Claude can write essential information from tool results to memory files before those results are cleared
- **Maintain long-running workflows:** Enable agentic workflows that would otherwise exceed context limits by offloading information to persistent storage
- **Access information on demand:** Claude can look up previously cleared information from memory files when needed, rather than keeping everything in the active context window

For example, in a file editing workflow where Claude performs many operations, Claude can summarize completed changes to memory files as the context grows. When tool results are cleared, Claude retains access to that information through its memory system and can continue working effectively.

To use both features together, enable them in your API request:

Python ▾



```
response = client.beta.messages.create(  
    model="claude-sonnet-4-5",  
    max_tokens=4096,  
    messages=[...],  
    tools=[  
        {  
            "type": "memory_20250818",  
            "name": "memory"  
        },  
        # Your other tools  
    ],  
    betas=["context-management-2025-06-27"],  
    context_management={  
        "edits": [  
            {"type": "clear_tool_uses_20250919"}  
        ]  
    }  
)
```

Client-side compaction (SDK)

Compaction is an SDK feature that automatically manages conversation context by generating summaries when token usage grows too large. Unlike server-side context editing strategies that clear content, compaction instructs Claude to summarize the conversation history, then replaces the full history with that summary. This allows Claude to continue working on long-running tasks that would otherwise exceed the context window.

How compaction works

When compaction is enabled, the SDK monitors token usage after each model response:

1. **Threshold check:** The SDK calculates total tokens as `input_tokens + cache_creation_input_tokens + cache_read_input_tokens + output_tokens`
2. **Summary generation:** When the threshold is exceeded, a summary prompt is injected as a user turn, and Claude generates a structured summary wrapped in `<summary></summary>` tags
3. **Context replacement:** The SDK extracts the summary and replaces the entire message history with it
4. **Continuation:** The conversation resumes from the summary, with Claude picking up where it left off

Using compaction

Add `compaction_control` to your `tool_runner` call:

Python ▾



```
client = anthropic.Anthropic()

runner = client.beta.messages.tool_runner(
    model="claude-sonnet-4-5",
    max_tokens=4096,
    tools=[...],
    messages=[
        {
            "role": "user",
            "content": "Analyze all the files in this directory and write a summary report"
        }
    ],
    compaction_control={
        "enabled": True,
        "context_token_threshold": 100000
    }
)

for message in runner:
    print(f"Tokens used: {message.usage.input_tokens}")

final = runner.until_done()
```

What happens during compaction

As the conversation grows, the message history accumulates:

Before compaction (approaching 100k tokens):

```
[
  { "role": "user", "content": "Analyze all files and write a report..." },
  { "role": "assistant", "content": "I'll help. Let me start by reading..." },
  { "role": "user", "content": [{ "type": "tool_result", "tool_use_id": "...", "content":
  { "role": "assistant", "content": "Based on file1.txt, I see..." }},
  { "role": "user", "content": [{ "type": "tool_result", "tool_use_id": "...", "content":
  { "role": "assistant", "content": "After analyzing file2.txt..." }},
  // ... 50 more exchanges like this ...
]
```

When tokens exceed the threshold, the SDK injects a summary request and Claude generates a summary. The entire history is then replaced:

After compaction (back to ~2-3k tokens):

```
    "role": "assistant",
    "content": "# Task Overview\nThe user requested analysis of directory files to produce a summary of the findings.\n\n# Summary\nThe analysis of the directory files revealed the following findings:\n\n1. The directory contains a large number of files, including many small files and a few large files.\n2. The files are organized into a hierarchical structure, with subdirectories and files nested within them.\n3. The files contain a variety of data, including text, images, and other types of content.\n4. The files are generally well-organized and easy to navigate.\n5. The files are stored in a secure location, and access is restricted to authorized users only.\n\n# Conclusion\nThe analysis of the directory files has identified a large number of files, organized into a hierarchical structure, containing a variety of data. The files are generally well-organized and easy to navigate, and are stored in a secure location with access restricted to authorized users only."
  }
]
```

Claude continues working from this summary as if it were the original conversation history.

Configuration options

Parameter	Type	Required	Default	Description
enabled	boolean	Yes	-	Whether to enable automatic compaction
context_token_threshold	number	No	100,000	Token count at which compaction triggers
model	string	No	Same as main model	Model to use for generating summaries
summary_prompt	string	No	See below	Custom prompt for summary generation

Choosing a token threshold

The threshold determines when compaction occurs. A lower threshold means more frequent compactions with smaller context windows. A higher threshold allows more context but risks hitting limits.

Python ▾



```
# More frequent compaction for memory-constrained scenarios
compaction_control={
    "enabled": True,
    "context_token_threshold": 50000
}

# Less frequent compaction when you need more context
compaction_control={
    "enabled": True,
    "context_token_threshold": 150000
}
```

You can use a faster or cheaper model for generating summaries:

Python ▾



```
compaction_control={
    "enabled": True,
    "context_token_threshold": 100000,
    "model": "claude-haiku-4-5"
}
```

Custom summary prompts

You can provide a custom prompt for domain-specific needs. Your prompt should instruct Claude to wrap its summary in `<summary></summary>` tags.

Python ▾



```
compaction_control={
    "enabled": True,
    "context_token_threshold": 100000,
    "summary_prompt": """Summarize the research conducted so far, including:
- Sources consulted and key findings
- Questions answered and remaining unknowns
- Recommended next steps

Wrap your summary in <summary></summary> tags."""
}
```

Default summary prompt

The built-in summary prompt instructs Claude to create a structured continuation summary including:

1. **Task Overview:** The user's core request, success criteria, and constraints
2. **Current State:** What has been completed, files modified, and artifacts produced
3. **Important Discoveries:** Technical constraints, decisions made, errors resolved, and failed approaches
4. **Next Steps:** Specific actions needed, blockers, and priority order
5. **Context to Preserve:** User preferences, domain-specific details, and commitments made

➤ [View full default prompt](#)

Limitations

Server-side tools

⚠ Compaction requires special consideration when using server-side tools such as [web search](#) or [web fetch](#).

When using server-side tools, the SDK may incorrectly calculate token usage, causing compaction to trigger at the wrong time.

For example, after a web search operation, the API response might show:

```
{
  "usage": {
    "input_tokens": 63000,
    "cache_read_input_tokens": 270000,
    "output_tokens": 1400
  }
}
```

The SDK calculates total usage as $63,000 + 270,000 = 333,000$ tokens. However, the `cache_read_input_tokens` value includes accumulated reads from multiple internal API calls made by the server-side tool, not your actual conversation context. Your real context length might only be the `63,000 input_tokens`, but the SDK sees 333k and triggers compaction prematurely.

Workarounds:

- Use the [token counting](#) endpoint to get accurate context length
- Avoid compaction when using server-side tools extensively

Tool use edge cases

When compaction is triggered while a tool use response is pending, the SDK removes the tool use block from the message history before generating the summary. Claude will re-issue the tool call after resuming from the summary if still needed.

Enable logging to track when compaction occurs:

Python ▾



```
import logging

logging.basicConfig(level=logging.INFO)
logging.getLogger("anthropic.lib.tools").setLevel(logging.INFO)

# Logs will show:
# INFO: Token usage 105000 has exceeded the threshold of 100000. Performing compaction.
# INFO: Compaction complete. New token usage: 2500
```

When to use compaction

Good use cases:

- Long-running agent tasks that process many files or data sources
- Research workflows that accumulate large amounts of information
- Multi-step tasks with clear, measurable progress
- Tasks that produce artifacts (files, reports) that persist outside the conversation

Less ideal use cases:

- Tasks requiring precise recall of early conversation details
- Workflows using server-side tools extensively
- Tasks that need to maintain exact state across many variables

	Powered by Claude	policy
Partners	Service partners	Terms of service:
Amazon Bedrock	Startups program	Commercial
Google Cloud's Vertex AI		Terms of service:
	Company	Consumer
	Anthropic	Usage policy
	Careers	
	Economic Futures	
	Research	
	News	
	Responsible Scaling Policy	
	Security and compliance	
	Transparency	